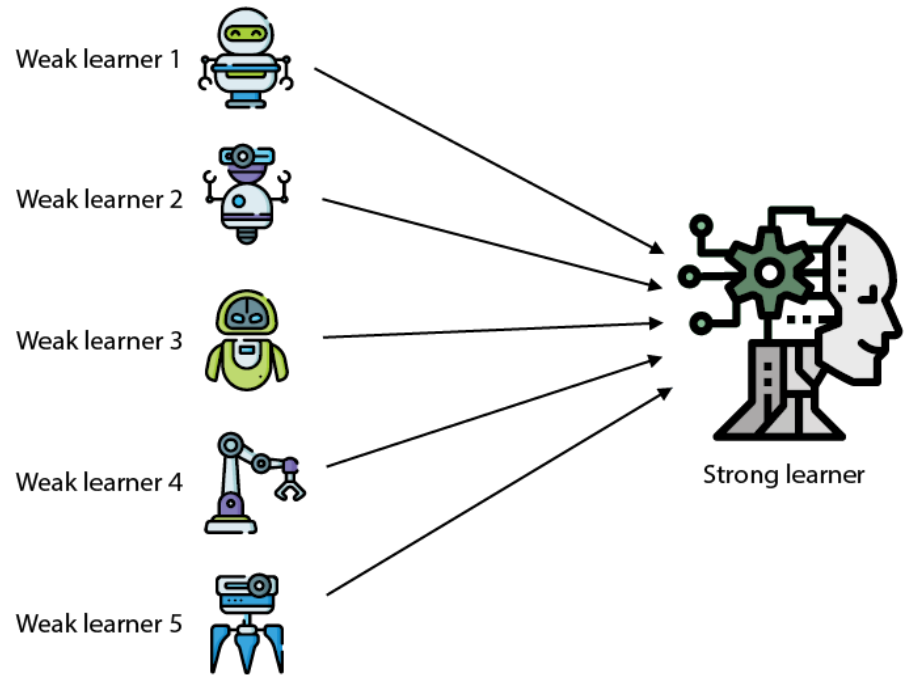
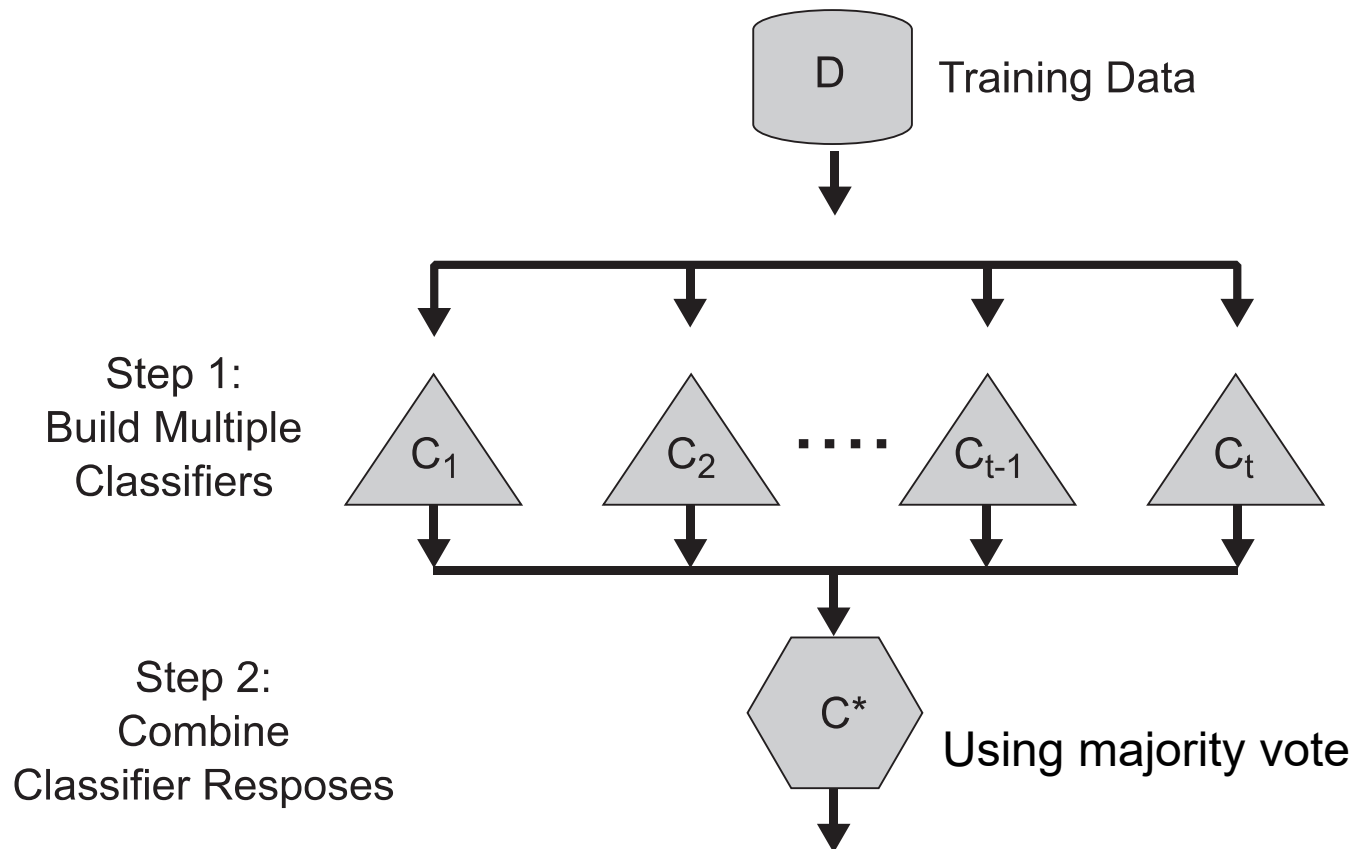


Ensemble Learning



General Approach of Ensemble Learning



- Basic Idea: Instead of learning one model, learn several and combine them
- Ensemble methods try to reduce the (overfitting) variance of complex models by *aggregating* responses of multiple base classifiers

Ensemble Methods

```
graph TD; A[Ensemble Methods] --> B[Bagging]; A --> C[Boosting]; A --> D[Stacking]; B --- B_desc[Homogeneous Learners<br/>Parallel Learning<br/>e.g. Random Forest]; C --- C_desc[Homogeneous Learners<br/>Sequential Learning<br/>e.g. Gradient Boosted Trees]; D --- D_desc[Heterogeneous Learners<br/>Parallel Learning];
```

Bagging

Homogeneous Learners
Parallel Learning
e.g. Random Forest

Boosting

Homogeneous Learners
Sequential Learning
e.g. Gradient Boosted Trees

Stacking

Heterogeneous Learners
Parallel Learning

Bagging (Bootstrap AGGREGatING)

Bootstrapping

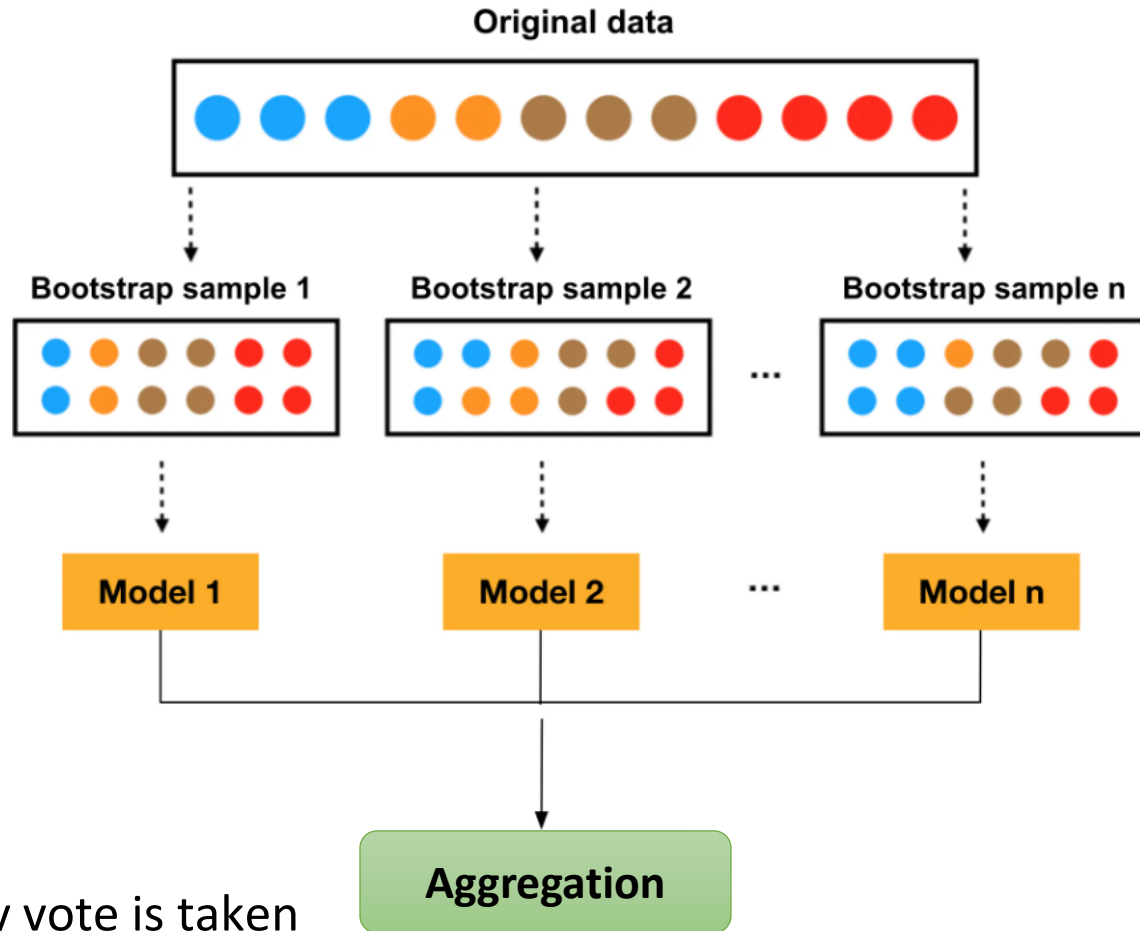
(sampling with replacement)

Model Training

(train a model on each bootstrap using one type of algorithm such as Decision Tree, Logistic Regression)

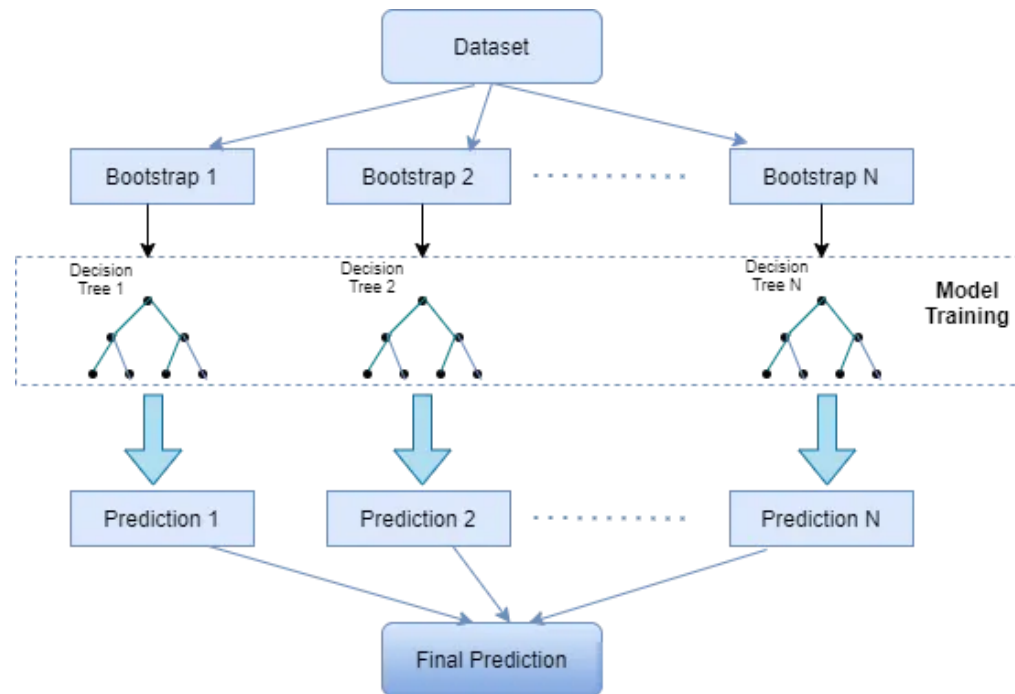
Aggregation

(Classification: Majority vote is taken
Regression: Average of outputs)



Random forest: an example implementation of bagging

- Construct multitude of decision trained on a random subset of the data and makes independent predictions
- The final prediction is determined by **aggregating** the predictions of individual trees, such as through a majority vote for classification tasks or averaging for regression tasks
- **Pros:**
 - Improve accuracy over single models; reduces overfitting (variance)
 - Easy to parallelize
- **Cons:** Difficult to interpret the resulting model



Random Forest

One Single Tree

```
In [27]:  ▶ #One Single Tree
          tree_clf = DecisionTreeClassifier(random_state=42)
          tree_clf.fit(X_train, y_train)
          print("Accuracy:", tree_clf.score(X_test, y_test))
```

Accuracy: 0.8716666666666667

Random Forest

```
In [28]:  ▶ randomForest_clf = RandomForestClassifier(n_estimators=100, random_state=42, max_samples=480)
          randomForest_clf.fit(X_train, y_train)
          print("Accuracy:", randomForest_clf.score(X_test, y_test))
```

Accuracy: 0.9166666666666666

Boosting

- Boosting allow **incrementally** building an ensemble
 - In each iteration, it sequentially train a **new model by focusing more on previously misclassified instances**
 - Initially, all **N** instances are assigned equal weights (equal chance for being selected for training)
 - For next round, the misclassified instances are assigned a **weight higher** than the correctly classified instances
 - Example 4 was misclassified in round 2
 - Its weight is increased. Therefore, it is more likely to be chosen again in subsequent rounds

Original Data	1	2	3	4	5	6	7	8	9	10
Boosting (Round 1)	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2)	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3)	4	4	8	10	4	5	4	6	3	4

Pros:

Reduces bias and variance

In some cases, boosting has been shown to yield better accuracy than bagging

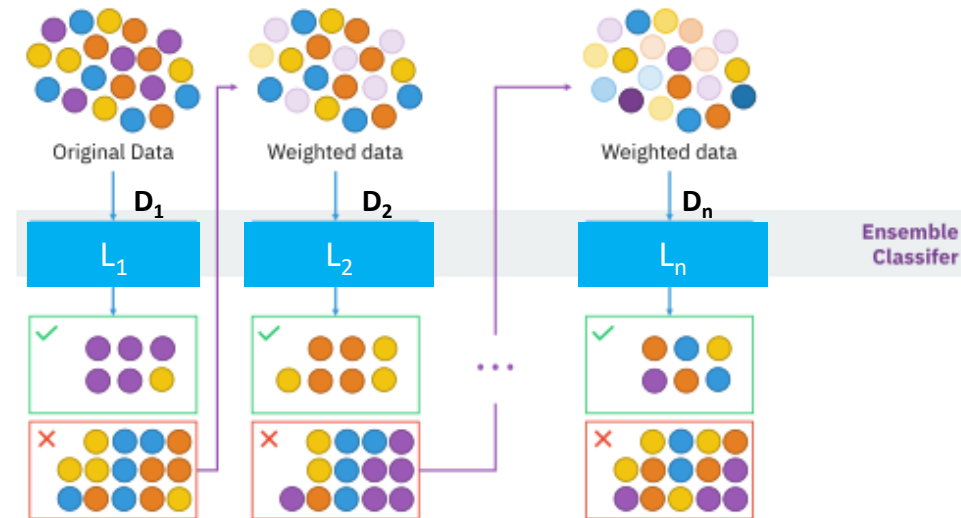
Cons:

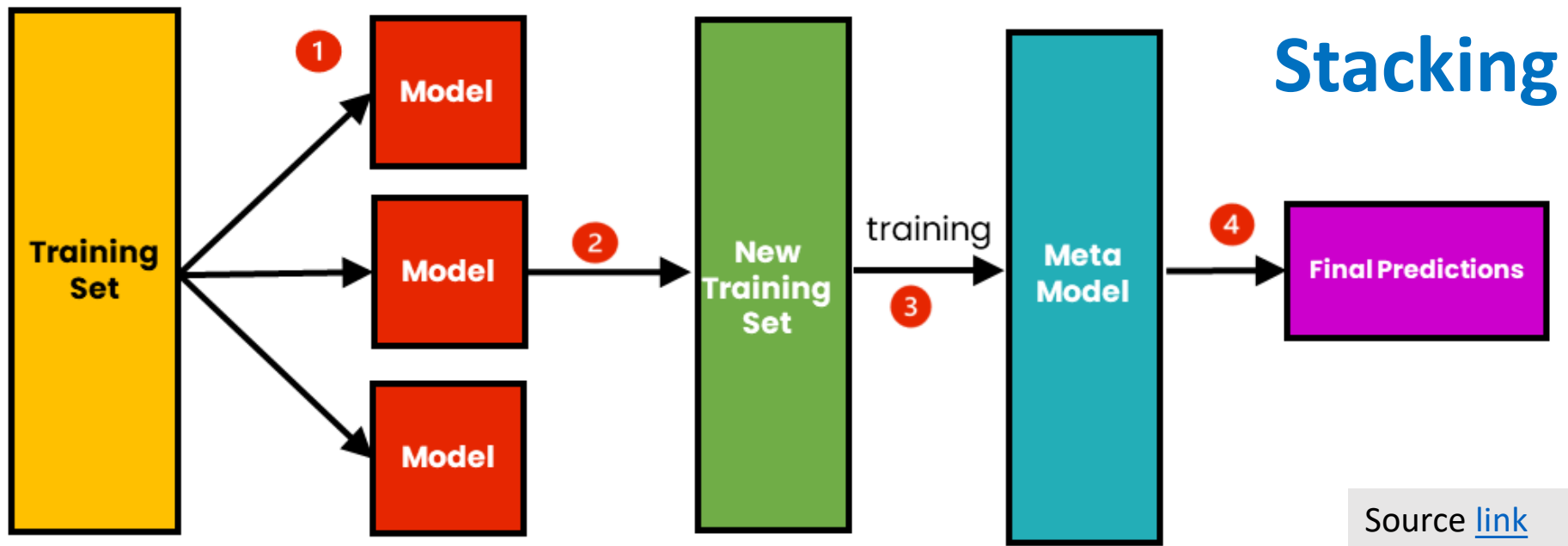
Tends to be more likely to overfit the training data

Sensitive to noisy data and outliers

Boosting

- **Equal weight** is given to the training data instances (D_1) at the **starting** round
 - (D_1) is then given to a base learner (L_1)
- The **mis-classified** instances by L_1 are assigned a weight higher than the correctly classified instances
 - This boosted data (D_2) is then given to second learner (L_2) and so on
- The results are then combined in the form of **weighted voting**, based on how **well a learner** performs





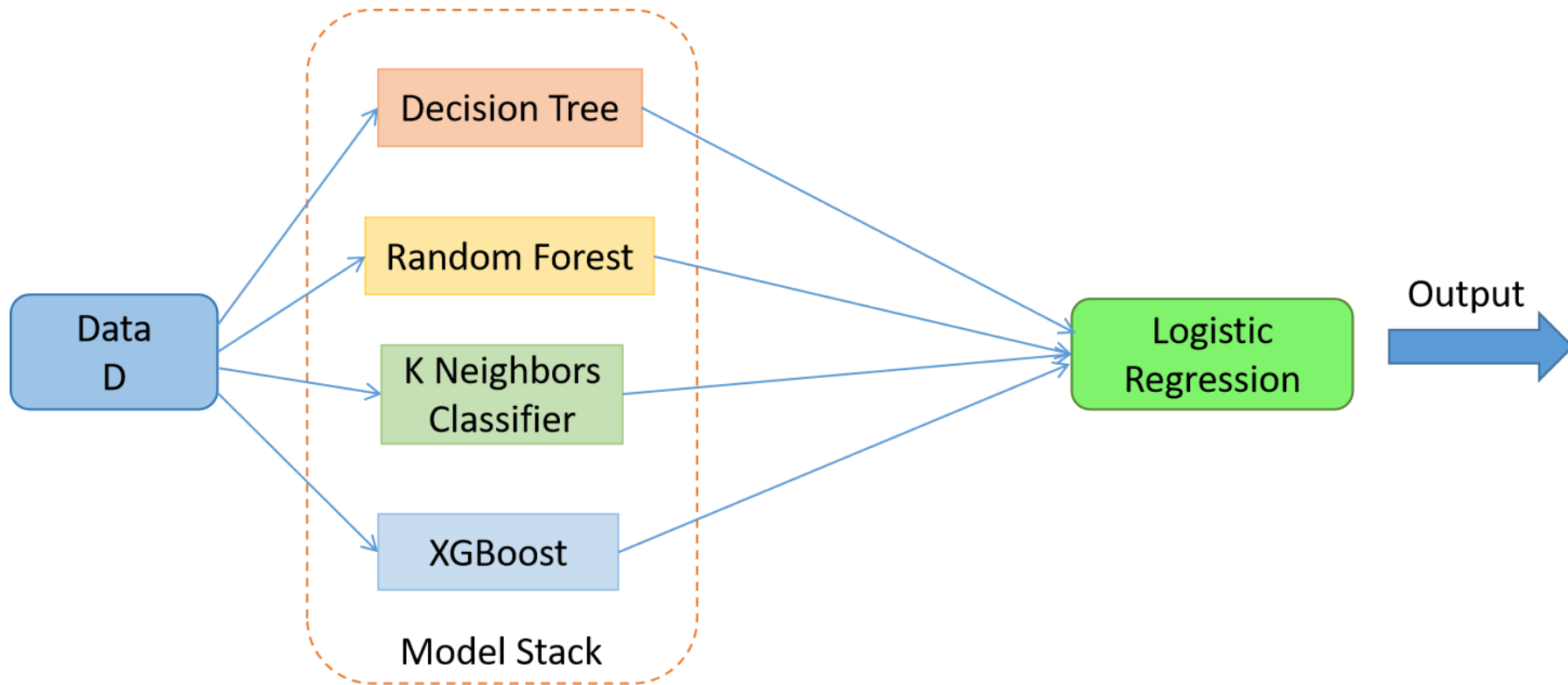
Stacking: training a ML model that combines the predictions of several base models trained using the same training data

1. **Base Models:** Train several diverse base models on the training data using different algorithms
2. **Generate Predictions:** Use each base model to make predictions on the validation set
3. **Meta-Model Training:** Combine the predictions from the base models and use them to train a meta-model (typically a simple model like logistic regression, or decision tree)
4. **Prediction:** Use the trained meta-model to make final predictions on unseen data

Stacking

- The main goal of stacking is to **improve the predictive performance** by leveraging the strengths of several base models, often yielding better performance than any single model in the ensemble
- It can help the model capture non-linear patterns and interactions between features that might be missed by individual models

Stacking Example



- Multiple machine learning algorithms (e.g., Decision Tree, XGBoost) are used as the base models trained on the same dataset
- Then we use the trained models to predict using the validation set
- The output of the base models on the validation set are then used to train the meta-model (Logistic regression in this example)